



THE IRON NEXUS

Railway Management System

Module-wise Data Structure Selection & Justification

Prepared By:

Rehan Ayub (24I-0631)
Malhan Bin Faisal (24I-0651)
Raja Moiz Khalid (24I-0674)

Section: CS-E

National University of Computer and Emerging Sciences (FAST-NUCES)
Islamabad Campus

May 12, 2026

Module-wise Data Structure Selection & Justification

Module	Data Structure(s)	Justification
1. Train Registry	AVL Tree (Self-Balancing BST)	Trains are keyed by unique TrainID. An AVL tree guarantees $O(\log n)$ insert, delete, and search even as the fleet grows to thousands of trains, because it self-balances after every operation. A plain BST could degrade to $O(n)$ in sorted-ID insertions. The balance factor is maintained via left/right rotations, keeping tree height minimal for emergency lookups.
2. Coach Management	Doubly Linked List	A train's coaches must support $O(1)$ insertion at front, at end, and after a given node, as well as removal of any coach and full reversal for return journeys. A doubly linked list satisfies all these needs: each Coach node holds both next and prev pointers, enabling bidirectional traversal (engine-to-caboose and back) and efficient mid-list insertion/deletion without shifting elements.
3. Rail Network	Weighted Undirected Graph + Dijkstra (Min-Heap Priority Queue)	Stations are vertices; tracks are weighted undirected edges stored in a 15×15 adjacency matrix. This allows $O(1)$ edge lookup and weight retrieval. Dijkstra's algorithm with a custom min-heap Priority Queue finds the shortest path in $O((V + E) \log V)$. The matrix also makes station/track removal trivial (zero out a row/column). Dynamic changes (add/remove station or track) are handled in-place.
4. Seat Booking	Hash Table (Layer 1) + BST (Layer 2)	Layer 1 — Hash Table: <code>seatNumber % 100</code> maps any seat directly to a memory slot for $O(1)$ average booking, cancellation, and status lookups. Collisions are resolved with chaining (linked list per bucket). Layer 2 — BST (SeatIndex): seats are inserted by seat number, giving an always-sorted structure. In-order traversal displays all seats, booked seats, or available seats in ascending order without any sorting step.
5. Operation Log	Stack (Array-based)	Every system action is pushed onto the Stack in the exact order it occurs. Because operators review recent actions first, a stack's LIFO (Last-In First-Out) property is ideal: <code>peek()</code> shows the latest entry instantly, <code>pop()</code> removes it, and <code>display()</code> iterates from top to bottom. An array-backed stack avoids pointer overhead and delivers $O(1)$ push/pop/peek.